

AIVIETNAM–AICOURSE2024

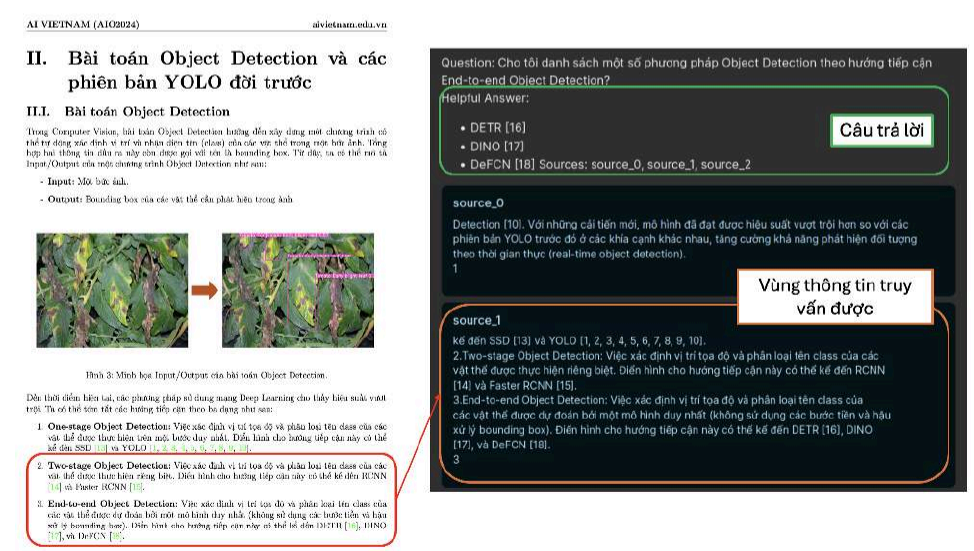
Ứng dụng RAG trong việc hỏi đáp tài liệu bài học AIO

Dinh-Thang Duong, Nguyen-Thuan Duong, Quang-Vinh Dinh

Ngày 23 tháng 6 năm 2024

I. Giới thiệu

Large Language Models (LLMs) (Tạm dịch: Các mô hình ngôn ngữ lớn) là loại mô hình cho phép người dùng nhập vào một văn bản với nội dung bất kỳ, thường là các yêu cầu hoặc câu hỏi. Từ đó, mô hình sẽ trả về câu trả lời dưới dạng văn bản thoả mãn yêu cầu của người dùng. Các ứng dụng phổ biến nhất về LLM có thể kể đến như ChatGPT, Gemini...



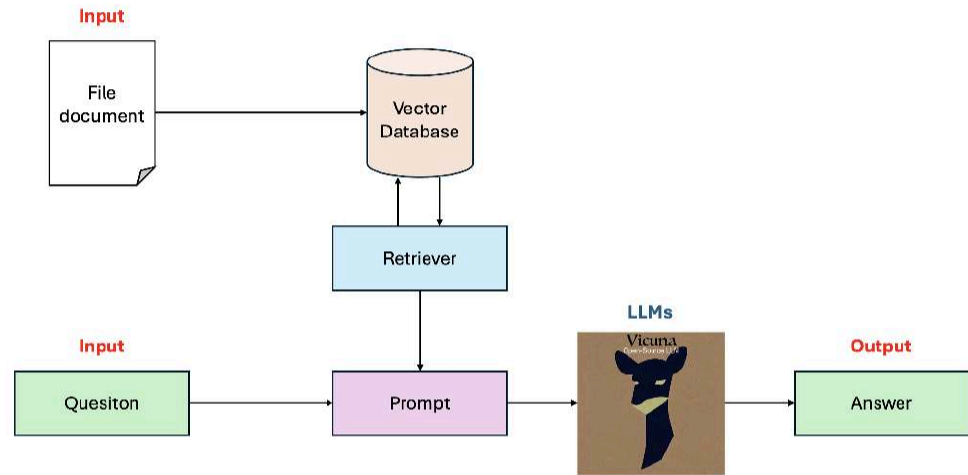
Hình 1: Sử dụng RAG trong việc hỏi đáp tài liệu về YOLOv10 trong AIO.

Trong LLMs, **Retrieval Augmented Generation (RAG)** là một kỹ thuật giúp LLM cải thiện chất lượng kết quả tạo sinh bằng cách tích hợp nội dung truy vấn và được tìm từ một nguồn tài

liệu này để trả lời cho một câu hỏi đầu vào. Trong project này, chúng ta sẽ tìm hiểu cách xây dựng một chương trình RAG cơ bản. Đồng thời, ứng dụng chương trình vào việc hỏi đáp tài liệu bài học trong khóa AIO. Thế đó, Input và Output của chương trình là:

- Input:** File tài liệu cần hỏi đáp và một câu hỏi liên quan đến nội dung tài liệu.
- Output:** Câu trả lời.

Tổng quan, luồng xử lý (pipeline) của chương trình RAG mà chúng ta sẽ xây dựng có dạng như sau:



Hình 2: Pipeline của chương trình RAG trong project.

II. Cài đặt chương trình

Trong phần này, chúng ta sẽ tìm hiểu cách cài đặt chương trình RAG theo hướng dẫn ở phần trước, bao gồm phần chương trình RAG và phần giao diện chat (optional). Cả hai phần đều sẽ được thực hiện trên Google Colab đã được kích hoạt GPU. **Lưu ý rằng, mục tiêu của project này hướng đến việc chiểu được một số khái niệm cơ bản về bài toán RAG trong AI. Vì vậy, các bạn có thể tạm thời không cần hiểu rõ các chức năng của các hàm, kỹ thuật sẽ được sử dụng trong bài.**

II.I. Chương trình RAG

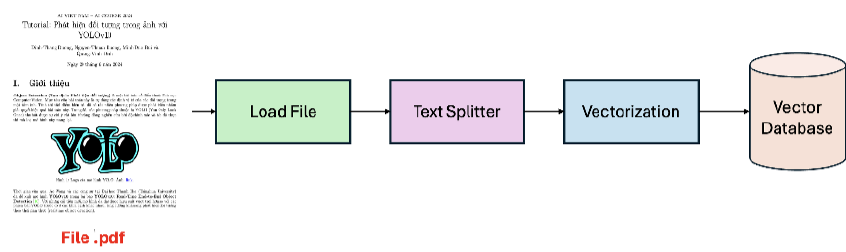
Trước tiên, chúng ta cần nắm được các bước xử lý cơ bản trong RAG bằng cách xây dựng một chương trình RAG đơn giản. Tại đây, ta sẽ sử dụng thư viện LangChain để thực hiện việc này. Các bước triển khai như sau:



Hình 3: Thư viện LangChain. Một thư viện được thiết kế chuyên biệt cho việc xây dựng các ứng dụng về LLMs. Ảnh: [link](#).

1. **Cài đặt các gói thư viện cần thiết:** Thực hiện cài đặt một số gói thư viện thông qua lệnh `pip install` như sau:
- ```
1! pip install -q transformers==4.41.2
2! pip install -q bitsandbytes==0.43.1
3! pip install -q accelerate==0.31.0
4! pip install -q langchain==0.2.5
5! pip install -q langchainhub==0.1.20
6! pip install -q langchain-chroma==0.1.1
7! pip install -q langchain-community==0.2.5
8! pip install -q langchain-huggingface==0.0.3
9! pip install -q python-dotenv==1.0.1
10! pip install -q pypdf==4.2.0
11! pip install -q numpy==1.24.4
```
2. **Xây dựng vector database:** Để thực hiện truy vấn, chúng ta cần phải có một cơ sở dữ liệu. Theo hướng dẫn của project, với dữ liệu nguồn là một file pdf, chúng ta sẽ thực hiện

đưacácnộidungtrongfilenàyvàocổsởdữliệu.Vềcác bướcthực hiện,cácbạncóthể  
coiquaảnhsau:



Hình4:Pipelinecác bướcthực hiệnxây dựngvectordatabase.

(a)Import các thư viện cần thiết:Đểxây dựngvectordatabase,chúngtacầnmột  
sốthưviệnsau:

```
1import torch
2
3from transformers import BitsAndBytesConfig
4from transformers import AutoTokenizer , AutoModelForCausalLM ,
 pipeline
5from langchain_huggingface import HuggingFaceEmbeddings
6from langchain_huggingface.llms import HuggingFacePipeline
7
8from langchain . memory import ConversationBufferMemory
9from langchain_community.chat_message_histories import
 ChatMessageHistory
10from langchain_community . document_loaders import PyPDFLoader ,
 TextLoader
11from langchain . chains import ConversationalRetrievalChain
12
13from langchain_chroma import Chroma
14from langchain_text_splitters import RecursiveCharacterTextSplitter
15from langchain_core . runnables import RunnablePassthrough
16from langchain_core.output_parsers import StrOutputParser
17from langchain import hub
```

(b)Đọc file pdf:Từmộtfilepdfchotrước(trongbàinàytasẽsử dụngfilemô tả  
làbàihướng dẫnYOLOv10,cácbạncóthể tảifilenàytạiđây),tasử dụngclass  
PyPDFLoaderđểđọcfilepdfnàylênnhưsau:

```
1Loader = PyPDFLoader
2FILE_PATH = " ./ AIO-2024- All-Materials . pdf "
3loader = Loader (FI LE_PATH)
4documents = loade r . load()
```

(c)Khởi tạo bộ tách văn bản (text splitter):Hầuhếtcác trường hợp,vùngthông  
tinmàchúngtacầnlàmột câunào đó trongfilevăn bản lớn.Vì vậy,sẽtốt hơn  
nếuchúngtatáchfilevăn bảnrathànhcácđoạnvăn bảnnhỏ,và mỗivăn bảnnhỏ  
nàytasẽcointưnlàmột tài liệu trongcổ sở dữ liệu.Dựavàoyếutưởngtrên,tasẽsử

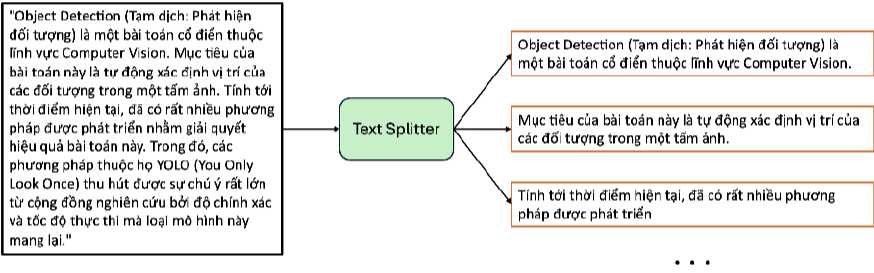
dùngclassTextSplitterđểthựcchiệntácvựtrên.Ởđây,takhaibáoinstancetext splitternhusau:

```
1text_splitter = RecursiveCharacterTextSp lit ter(chunk_size=1000 ,
2chunk_overlap =100)
```

Vớitext\_splitter,tathựcchiệntáchfilepdf:

```
1docs = text_splitter .spli t_ doc ument s (documents)
2
3print ("Nu mber of sub-do cument s : " ,len(docs))
4print (docs [0])
```

Ởhaidòngprinttrên,tainsốlượngtàiliệusaukhítáchkhỏipdfvàinnộidungtài liệuđầutiên.



Hình5:Minhhọaviệctáchvăn bảnlớnthànhcác văn bảnnhỏhơn.

(d)**Khởi tạo instance vectorization:**Các văn bản gốc được biểu diễn dưới dạng string.

Nếu giữ dạng biểu diễn này, việc truy vấn sẽ gặp khó khăn và kết quả truy vấn không được chính xác. Để khắc phục việc này, chúng ta có thể thực hiện chuyển đổi các văn bản thành các vector. Trong file notebook, tach đoạn codesau:

```
1embedding = HuggingFaceEmbeddings ()
```

embedding instancesẽgiúpchúngtathựcchiệnviệchchuyểnđổivăn bản thànhvector.

(e)**Khởi tạo vector database:**Vớicáchthôngtinvềdanhsáchcácdocuments(từkết quảtext\_splitter vàobjectdùngđểvectorizationembedding,tasẽđưavàohàmđùng đểkhởitạovectordatabasenhusau:

```
1vector_db = Chroma .f ro m_ do cu me nt s (do cu me nt s = docs ,
2embedding = embedding)
3
4retriever = vector_db . a s _re tri ever ()
```

Tacóthểthửthựcchiệntruy vấnvớimộtđoạnvăn bảnbất kitạidây:

```
1result = retriever .invoke (" What is YOLO ?")
2
3print ("Nu mber of r el ev an t d oc um ents: " ,len(result))
```

Kết quả của đoạn code này sẽ trả về cho ta danh sách các tài liệu có liên quan đến đoạn văn bản đầu vào.

3.Khởi tạo mô hình ngôn ngữ lớn: Trong project này, ta sẽ sử dụng mô hình Vicuna, một mô hình ngôn ngữ lớn nguồn mở có hiệu suất rất tốt, có thể phản hồi tốt với tiếng Việt. Các bạn có thể đọc thêm về mô hình Vicuna tại [đây](#).



Hình 6: Mô hình Vicuna. Ảnh: [link](#).

Một vấn đề của mô hình ngôn ngữ lớn đó là nó yêu cầu tài nguyên về phần cứng khá lớn. Vì vậy, để khởi tạo được mô hình Vicuna trên Colab, chúng ta sẽ cần thực hiện một số bước sau:

(a) Khai báo một số cài đặt cần thiết cho mô hình:

```
1 nf4_config = BitsAndBytesConfig (
2 load_in_4bit=True,
3 bnb_4bit_quant_type="nf4",
4 bnb_4bit_use_double_quant=True,
5 bnb_4bit_compute_dtype = torch.bfloat16
6)
```

(b) Khởi tạo mô hình và tokenizer:

```
1 MODEL_NAME = "lmsys / vicuna-7b - v1.5 "
2
3 model = AutoModelForCausalLM . from_pretrained (
4 MODEL_NAME ,
5 quantization_config = nf4_config ,
6 low_cpu_mem_usage=True
7)
8
9 tokenizer = AutoTokenizer . from_pretrained (model_name)
```

(c) Tích hợp tokenizer và model thành một pipeline để tiện sử dụng:

```
1 model_pipeline = pipeline (
2 "text-generation" ,
3 model = model ,
4 tokenizer = tokenizer ,
```

```

6max_new_tokens=512,
7pad_token_id=tokenizer . eos_token_id ,
8device_map = " auto "
9)
10llm = HuggingFacePipeline (
11pipeline = model_pipeline ,
12)
```

4.Chạy chương trình:Vớivectordatabase,retrieveryàomôhìnhVicunadãhoànthiện.Ta sẽkêthợpchúnglạiđểhoànthànhchươngtrìnhRAGđầutiêncủamình.Cácbạncóthể testthửbằngcáchđặtcáccâuhoicóliênquandếnfiletàiliệu,câutrảlờicủamôhìnhsẽ nằmởphần"Answer:"trongoutput.

```

1prompt = hub . pull (" rlm / rag- prompt ")
2
3def format_docs (docs) :
4return "\n\n". join (doc . page_content for doc in docs)
5
6rag_chain = (
7["context": retriever | format_docs , " question ": RunnablePassthrough
8() }
9| prompt
10| llm
11)
12
13USER_QUESTION = " YOLOv10 là gì ?"
14output = rag_chain . invoke (USER_QUESTION)
15answer = output . split ('Answer:') [1].strip()
16print (answer)
```

```

Human: You are an assistant for question-answering tasks. Use the following pieces of retrieved context
to answer the question. If the question is not answerable, just say "I don't know".
Question: YOLO là gì?
Context: Quan sát kết quả trên, ta có thể thấy trên cùng một phiên bản, YOLOv18 có mức độ tối ưu tốt
hơn về mặt tham số mô hình cũng như độ trễ trong inference trong khi vẫn giữ được độ chính
xác ngang hoặc hơn so với các phiên bản trước.
18

Khác.
-Ưu điểm : Cân bằng tốt giữa tốc độ và độ chính xác, dễ dàng sử dụng và triển khai.
-Nhược điểm : Yêu cầu phần cứng mạnh để đạt hiệu năng tối ưu.
5

AI VIETNAM (AIO2024) aivietnam.edu.vn
Hình 8: PSI và các kiến trúc tương tự. Ảnh: [9].
-Điểm mới : YOLOv9 sử dụng PSI và GELAN để cải thiện độ chính xác và hiệu suất của
mô hình.
-Ưu điểm :
+ Kiến trúc tiên tiến: Sử dụng PSI và GELAN giúp mô hình duy trì thông tin quan
trọng và tối ưu hóa quá trình huấn luyện, làm cho YOLOv9 trở nên mạnh mẽ và linh
hoạt hơn trong nhiều ứng dụng khác nhau.
+ Tốc độ nhanh hơn: YOLOv9 có thể xử lý hình ảnh nhanh hơn so với YOLOv8 nhờ
vào các cải tiến trong kiến trúc mạng.
-Nhược điểm :
+ Mặc dù nhanh hơn YOLOv8, YOLOv9 vẫn yêu cầu nhiều tài nguyên tính toán, đặc
biệt là khi xử lý hình ảnh độ phân giải cao.
...
phiên bản YOLO trước đó ở các khía cạnh khác nhau, tăng cường khả năng phát hiện đối tượng
theo thời gian thực (real-time object detection).
1
Answer: YOLO là một hệ thống mô hình máy học được sử dụng cho việc phát hiện đối tượng trong hình ảnh.
```

Hình7:Minhhọakếtquảđầuracủachươngtrình.

Như vậy, chúng ta đã hoàn thành một chương trình Python về RAG, có khả năng trả lời các câu hỏi dựa trên nội dung trong một file pdf.

## II.II. Xây dựng giao diện chat

Sau khi có chương trình RAG có thể trả về output đúng như dự định, ta có thể tích hợp vào với một giao diện chat để có được một ứng dụng chat hoàn chỉnh. Để xây dựng phần giao diện trong project này, chúng ta sẽ sử dụng thư viện Chainlit. Lưu ý rằng, phần code Chainlit có sử dụng những đoạn code Python phức tạp, các bạn mới chỉ cần xem và chạy dưới dạng blackbox.



Hình 8: Chainlit, một thư viện hỗ trợ giao diện cho các ứng dụng về chatbot. Ảnh: [link](#).

Các bước thực hiện như sau:

### 1. Tải các gói thư viện:

```
1! pip install -q transformers==4.41.2
2! pip install -q bitsandbytes==0.43.1
3! pip install -q accelerate==0.31.0
4! pip install -q langchain==0.2.5
5! pip install -q langchainhub==0.1.20
6! pip install -q langchain-community==0.1.1
7! pip install -q langchain-openai==0.1.9
8! pip install -q langchain-huggingface==0.0.3
9! pip install -q chainlit==1.1.304
10! pip install -q python-dotenv==1.0.1
11! pip install -q pydantic==4.2.0
12! npm install -g localtunnel
13! pip install -q numpy==1.24.4
```

### 2. Import các gói thư viện cần thiết:

```
1import chainlit as cl
2import torch
3
4from chainlit.types import AskFileResponse
5
6from transformers import BitsAndBytesConfig
7from transformers import AutoTokenizer, AutoModelForCausalLM, pipeline
8from langchain_huggingface.llms import HuggingFacePipeline
9
10from langchain.memory import ConversationBufferMemory
11from langchain_community.chat_message_histories import ChatMessageHistory
```

```
12 from langchain . chains import ConversationalRetrievalChain
13
14 from langchain_huggingface import HuggingFaceEmbeddings
15 from langchain_chroma import Chroma
16 from langchain_community . document_loaders import PyPDFLoader , TextLoader
17 from langchain_text_splitters import RecursiveCharacterTextSplitter
18 from langchain_core . runnables import RunnablePassthrough
19 from langchain_core.output_parsers import StrOutputParser
20 from langchain import hub
```

3.Cài đặt lại các hàm và instance ở file trước: Chúng ta khai báo lại một số hàm và instance ở phần trước. Các bạn có thể quay lại mục trước để xem phần giải thích:

```
1 text_splitter = RecursiveCharacterTextSplitter(chunk_size=1000 ,
2 chunk_overlap=100)
3
4 embedding = HuggingFaceEmbeddings ()
```

4.Xây dựng hàm xử lý file input đầu vào: Để thuận tiện trong việc cài đặt, tagom việc đọc và tách văn bản vào chung một hàm như sau:

```
1 def process_file(file: AskFileResponse):
2 if file .type == " text / plain " :
3 loader = TextLoader
4 elif file.type == " application / pdf " :
5 loader = PyPDFLoader
6
7 loader = Loader(file . path)
8 documents = loader . load()
9 docs = text_splitter . split_documents (documents)
10 for i , doc in enumerate (docs) :
11 doc . metadata[" source "] = f " source_ {i } "
12 return docs
```

5.Xây dựng hàm khởi tạo Chroma database: Tương tự phát trên, ta xây dựng hàm khởi tạo vector database cho phần cài đặt này:

```
1 def get_vector_db (file: AskFileResponse):
2 docs = process_file(file)
3 cl.user_session.set (" docs " , docs)
4 vector_db = Chroma . from_documents (documents = docs ,
5 embedding = embedding)
6 return vector_db
```

Tại line 2, gọi hàm process\_file() để xử lý file input và trả về các tài liệu nhỏ (docs). Sau đó, khởi tạo Chroma vector database bằng cách gọi Chroma.from\_documents() và truyền vào docs cũng như embedding đã khởi tạo trước đó.

6.Khởi tạo mô hình ngôn ngữ lớn:

```
1 def get_huggingface_llm (model_name : str = " lmsys / vicuna-7 b - v1 .5 " ,
2 max_new_token : int = 512) :
3 nf4_config = BitsAndBytesConfig (
4 load_in_4bit=True ,
5 bnb_4bit_quant_type=" n f4 " ,
```

```
6bnb_4bit_use_double_quant=True,
7bnb_4bit_compute_dtype = torch . bfloat16
8)
9model = AutoModelForCausalLM . from_pretrained (
10 model_name ,
11 quantization_config = nf4_config ,
12 low_cpu_mem_usage=True
13)
14 tokenizer = AutoTokenizer . from_pretrained (model_name)
15
16 model_pipeline = pipeline (
17 "text-generation " ,
18 model = model ,
19 tokenizer = tokenizer ,
20 max_new_tokens = max_new_token ,
21 pad_token_id= tokenizer . eos_token_id ,
22 device_map = " auto "
23)
24
25 llm = HuggingFacePipeline (
26 pipeline = model_pipeline ,
27)
28 return llm
29
30 LLM = get_huggingface_llm ()
```

7.Khởi tạo welcome message:

```
1welcome_message = """Welcome to the PDF QA! To get started:
21. Upload a PDF or text file
32. Ask a question about the file
4"""
```

8.Khởi tạo hàm on\_chat\_start:

```
1@cl . on_chat_start
2async def on_chat_start() :
3files = None
4while files is None :
5files = await cl . AskFileMessage(
6content = welcome_message ,
7accept=[" text / plain " , " application / pdf "] ,
8max_size_mb=20 ,
9timeout=180 ,
10). send()
11file = files [0]
12
13msg = cl . Message (content=f " Processing '{ file . name }' ... " ,
14disable_feedback=True)
15await msg . send()
16
17vector_db = await cl . make_async (get_vector_db) (file)
18
19message_history = ChatMessageHistory ()
20memory = ConversationBufferMemory(
```

```
21 memory_key = "chat_history",
22 output_key = "answer",
23 chat_memory = message_history,
24 return_messages = True,
25)
26 retriever = vector_db.as_retriever(search_type="mmr",
27 search_kwargs={'k': 3})
28
29 chain = ConversationRetrievalChain.from_llm(
30 llm = LLM,
31 chain_type = "stuff",
32 retriever = retriever,
33 memory = memory,
34 return_source_documents = True
35)
36
37 msg.content = f"{{ file.name }} processed. You can now ask questions!"
38 await msg.update()
39
40 cl.user_session.set("chain", chain)
```

9.Khởi tạo hàm on\_message:

```
1 @cl.on_message
2 async def on_message(message: cl.Message):
3 chain = cl.user_session.get("chain")
4 cb = cl.AsyncLangChainCallbackHandler()
5 res = await chain.ainvoke(message.content, callbacks=[cb])
6 answer = res["answer"]
7 source_documents = res["source_documents"]
8 text_elements = []
9
10 if source_documents:
11 for source_idx, source_doc in enumerate(source_documents):
12 source_name = f"source_{source_idx}"
13 text_elements.append(
14 cl.Text(content=source_doc.page_content,
15 name=source_name)
16)
17 source_names = [text_el.name for text_el in text_elements]
18
19 if source_names:
20 answer += f"\nSources: {', '.join(source_names)}"
21 else:
22 answer += "\nNo sources found"
23
24 await cl.Message(content=answer, elements=text_elements).send()
```

10.Chạy chainlit app:

```
1 !chainlit run app.py --host 0.0.0.0 --port 8000 & >/content/logs.txt
```

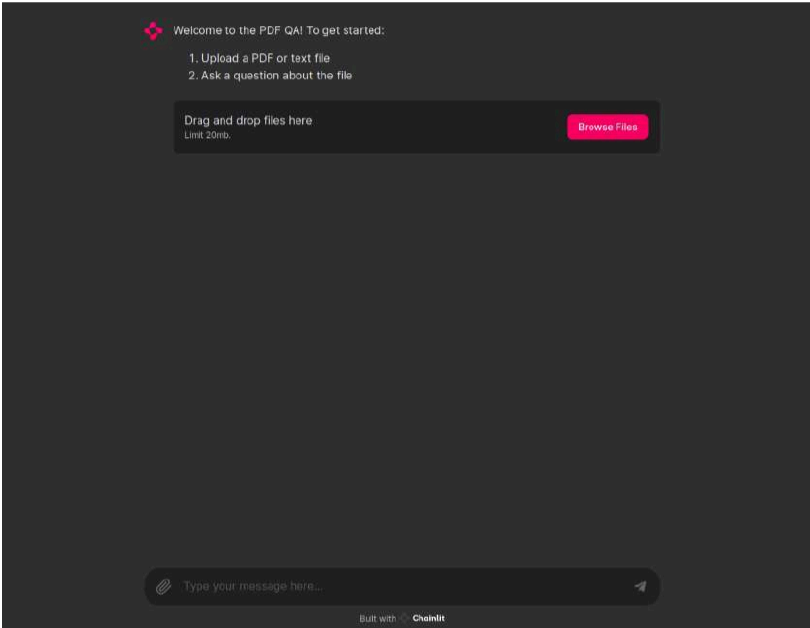
11.Expose localhost thành public host bằng local tunnel:

```
1 import urllib
```

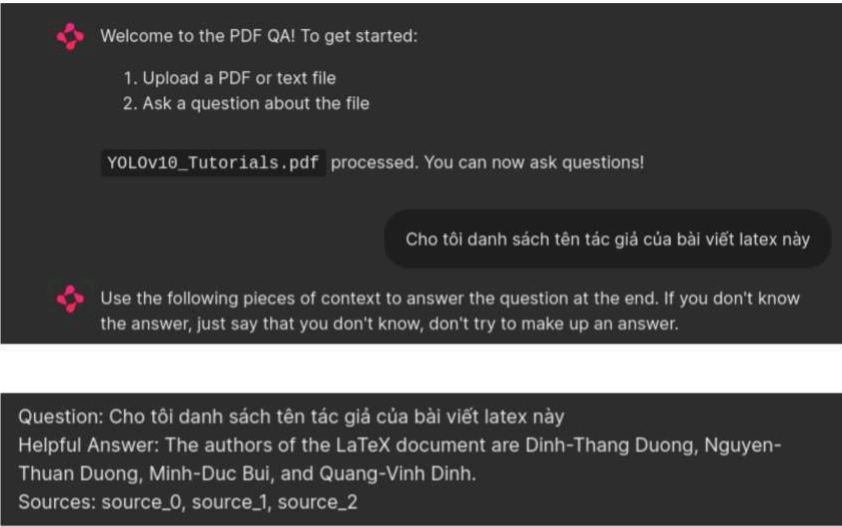
```
print (" Password / Enpoint IP for localtunnel is :",urllib . request . urlopen ('
https ://ipv4 . icanhazip . com ') . read() . d e c o d e (' u t f 8 ') . strip ("\n"))

! ! l t - - p o r t 80 00 --s u b d o m a i n a i v n - s i m p l e - r a g
```

Khichạyxongcácbuốctrên,cácbantruy cậpvàodường dẫnlocaltunnel,nhập passwordđược cung cấp để truy cập vào giaodienweb. Lưu ý rằng, phương án sử dụng localtunnellàm một phương án truy cập vào localhost trên Colab đơn giản nhưng có thể có lúc không hoạt động được. Nếu có máy tính có GPU, các bạn có thể đưa code này chạy ở máy để tiện sử dụng và đảm bảo lâu hơn.



Hình9:Giaodiệnchattrongchươngtrình.



Hình10:Thử nghiệm một đoạn chat trên giao diện chúng ta vừa hoàn thành.

### III. Câu hỏi trắc nghiệm

1. RAG là viết tắt của cụm từ nào sau đây trong ngữ cảnh mô hình ngôn ngữ lớn (LLM)?

(a) Randomized Augmentation Generator.

(b) Retrieval Augmented Generation.

(c) Recurrent Adaptive Generator.

(d) Recursive Algorithmic Generator.
2. Kỹ thuật RAG cải thiện mô hình ngôn ngữ lớn bằng cách nào ?

(a) Truy vấn thêm các thông tin bên ngoài.

(b) Dùng google search.

(c) Cải thiện việc sai chính tả.

(d) Bỏ dấu câu.
3. Input và Ouput trong hệ thống RAG cho bài toán PDF Question-Answering là gì?

(a) Input là video, Ouput là text.

(b) Input là text, Ouput là text.

(c) Input là image và text, Ouput là text.

(d) Input là pdf và text, Ouput là text.
4. Trước khi sinh ra câu trả lời, RAG sẽ làm gì ?

(a) Tạo một chuỗi text ngẫu nhiên.

(b) Dịch câu hỏi đầu vào sang một ngôn ngữ khác.

(c) Truy vấn các thông tin liên quan đến câu hỏi.

(d) Kiểm tra chính tả của câu hỏi đầu vào.
5. Trong RAG, vì sao lại cần chuyển đổi các đoạn văn bản thành vector?

(a) Để giảm chi phí tính toán trong quá trình ánh xạ văn bản vào không gian vector.

(b) Để tăng tính ngẫu nhiên và sáng tạo trong quá trình sinh câu trả lời.

(c) Để tăng tốc độ xử lý và giảm thời gian huấn luyện mô hình.

(d) Để mô hình có thể hiểu và xử lý thông tin dưới dạng số học, dễ dàng tính toán và so sánh.
6. Chức năng của đoạn code dưới đây là gì ?

```
from langchain_text_splitters import RecursiveCharacterTextSplitter
```

```
text_splitter = RecursiveCharacterTextSplitter(chunk_size=1000, chunk_overlap=100)
```

- (a) Khởi tạo bộ chia text.

- (b) Khởi tạo bộ chia vector.

(c) Import text từ langchain.

(d) Truy vấn text.
7. Tại sao cần phải có vector database trong hệ thống RAG?

(a) Giúp hệ thống chạy nhanh hơn.

(b) Làm cho hệ thống bảo mật cao hơn.

(c) Thuận tiện cho việc truy vấn thông tin có liên quan.

(d) Lưu trữ hiệu quả.
8. Cho đoạn code sau, chức năng của `embedding` là gì?

```
from langchain_huggingface import HuggingFaceEmbeddings
embedding = HuggingFaceEmbeddings()
```

(a) Làm mô hình LLM.

(b) Dùng để chuyển văn bản thành vector.

(c) Dùng để truy vấn thông tin văn bản.

(d) Làm một vector database.
9. Chainlit là gì?

(a) Là một thư viện giúp xây dựng các ứng dụng hội thoại với AI.

(b) Là một phần mềm chatbot.

(c) Là giao diện tích hợp chat GPT.

(d) Là một thư viện để huấn luyện AI.
10. Câu lệnh nào sau đây dùng để khởi chạy chainlit?

(a) `run chainlit`

(b) `chainlit run --app.py`

(c) `chainlit run`

(d) `chainlit run app.py`
- 15